

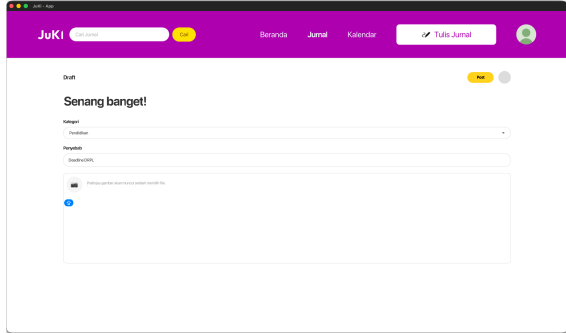
Dipersiapkan oleh: K02/GO2

- Nadine Arindy Octavia - 18224012
- Danya Damaris - 18224024
- Anisa Aulia Alhaqi - 18224080
- Zahrah Nur Azizah - 18224092
- Riko Satriya Giovanni - 18224108

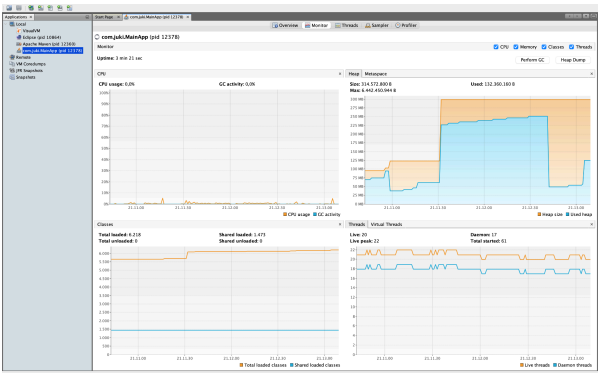
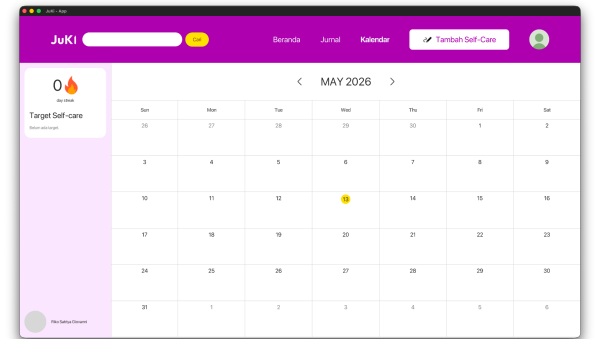
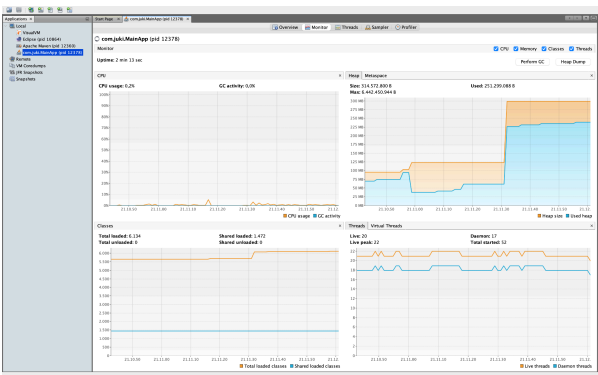
Performance Optimization

Lakukan identifikasi *bottleneck* performa pada fitur yang berat menggunakan *profiling tools*.

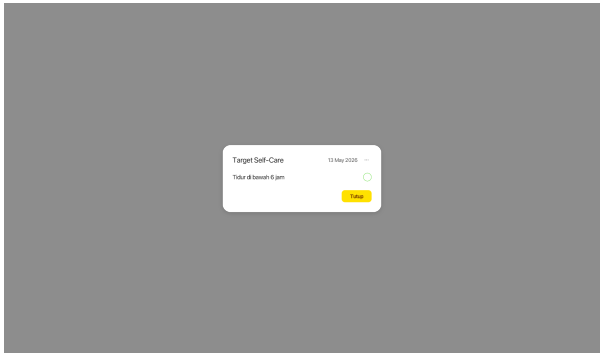
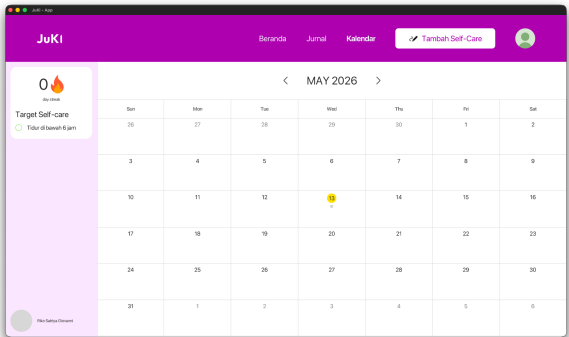
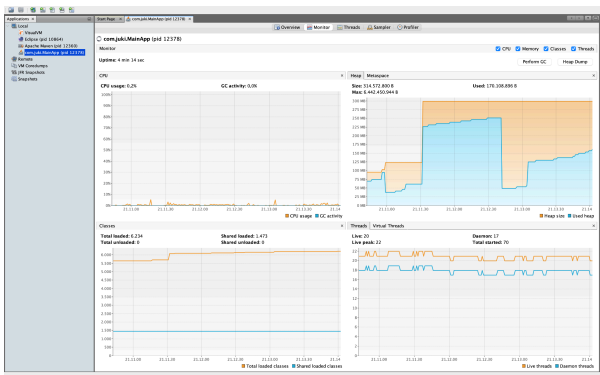
Untuk Java: Gunakan VisualVM (bawaan JDK) atau JProfiler (jika tersedia) untuk memantau penggunaan memori (Heap) dan CPU *usage*.

Fitur	SS
Menulis Jurnal	

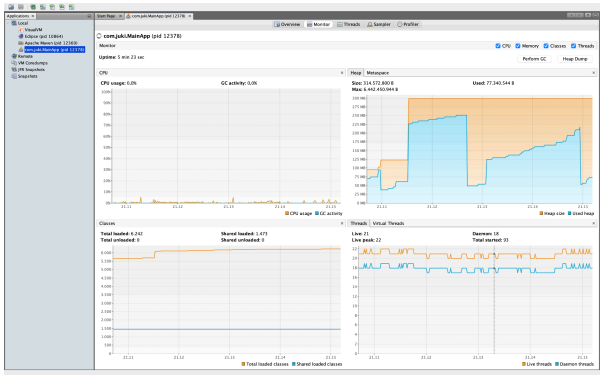
Masuk Ke Kalendar

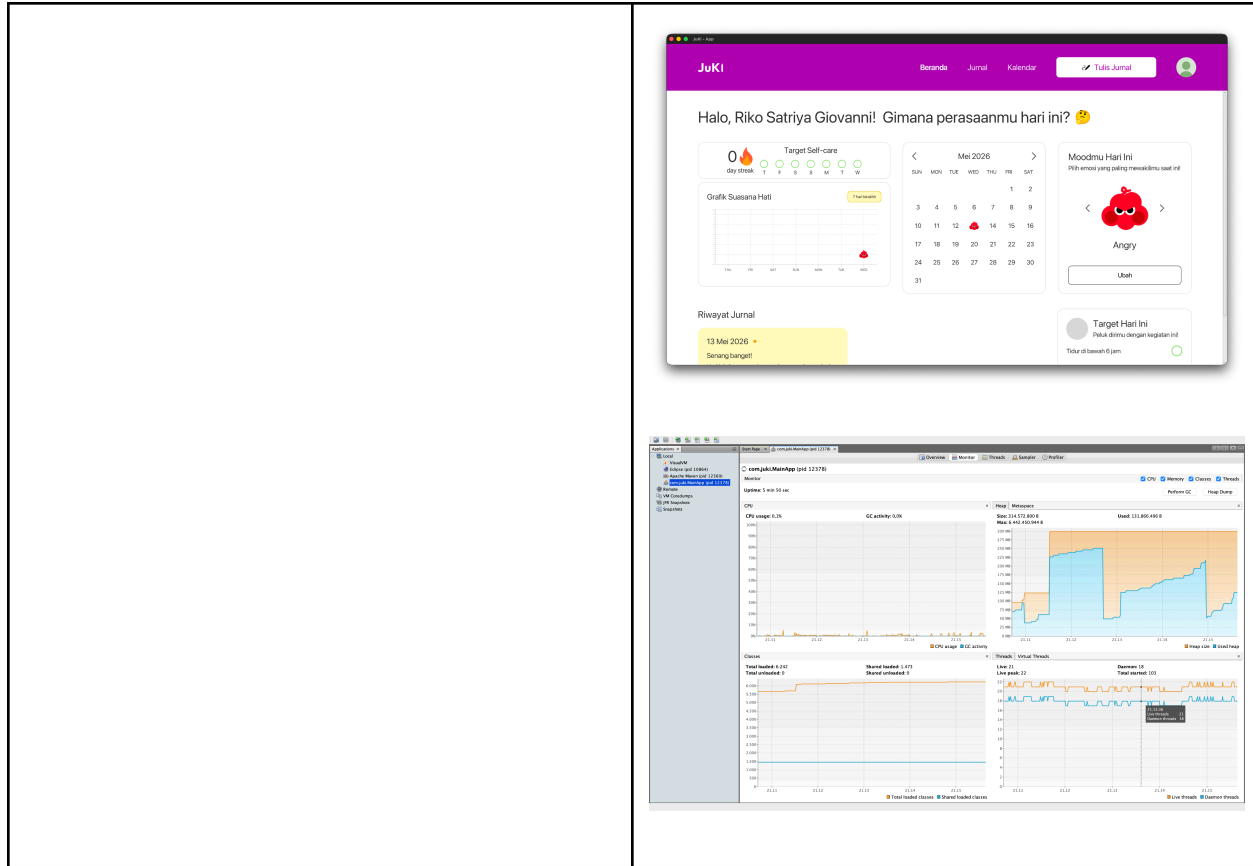


Memodifikasi Self Target



Mengubah Mood Harian





Pada Bagian bagian krusial tersebut:

1. Menulis Jurnal (Fitur EntryFormView)

Pada bagian ini, lonjakan memori dan CPU terjadi karena beberapa proses manajemen data:

- Pemrosesan Gambar: Saat kamu mengunggah foto jurnal, aplikasi melakukan penyalinan file fisik menggunakan Files.copy dan menghasilkan UUID baru. Proses ini memicu aktivitas CPU untuk operasi I/O.
- Rendering Pratinjau: Penggunaan ImageView untuk menampilkan pratinjau foto dalam FlowPane memakan memori Heap karena objek gambar dimuat ke dalam memori aplikasi sebelum disimpan ke database.
- Penyimpanan Database: Saat menekan tombol "Post", EntryController melakukan transaksi database yang melibatkan tabel JournalEntry dan Photo secara bersamaan, yang meningkatkan penggunaan CPU sesaat.

2. Masuk Ke Kalendar (Fitur CalendarView & DashboardView)

Kenaikan resource pada fitur kalender disebabkan oleh intensitas kueri:

- Kueri Mood Bulanan: Aplikasi memanggil `moodController.getMoodsByMonth` untuk mengambil semua data emosi dalam satu bulan. Jika data jurnal sudah banyak, pemrosesan *ResultSet* di *EntryController* menjadi *bottleneck* yang membebani CPU.
- Rendering Grid: Pembuatan objek *StackPane* dan *Label* secara dinamis untuk setiap tanggal (30+ sel) di dalam *GridPane* setiap kali bulan berganti menyebabkan alokasi objek baru di memori Heap.

3. Memodifikasi Self Target (Fitur GoalModal)

Aktivitas pada bagian ini memicu pembaruan UI secara *real-time*:

- Data Binding & Cache: Penggunaan *GoalService* yang menyimpan cache tujuan (*Goals Cache*) menyebabkan aktivitas memori saat data diperbarui.
- UI Refresh: Karena terdapat *listener* pada *goalsCache*, setiap perubahan target akan memicu *Platform.runLater* untuk membangun ulang widget *streakWidget* dan *dailyTargets*, yang menyebabkan lonjakan CPU kecil untuk kalkulasi ulang *streak*.

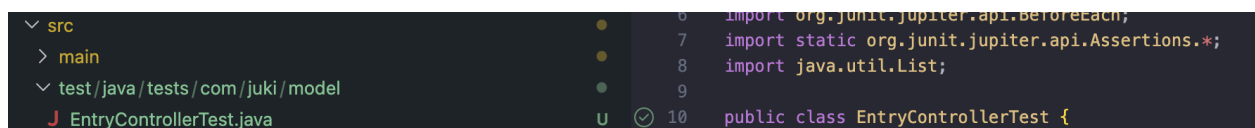
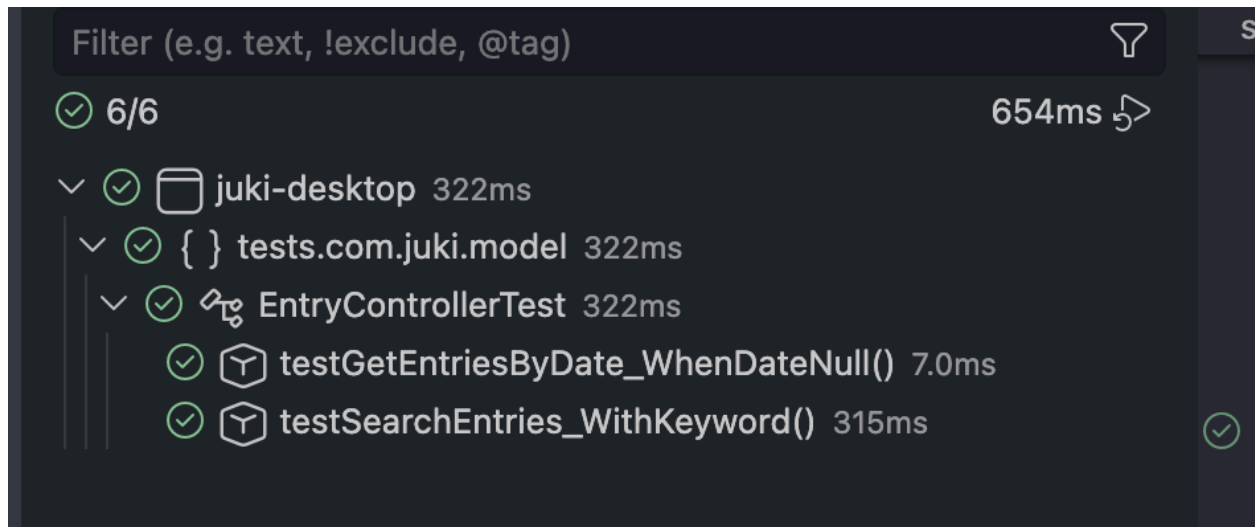
4. Mengubah Mood Harian (Fitur MoodSelector)

Kenaikan pada bagian ini didominasi oleh manajemen aset visual:

- Aset Emosi: Saat menggeser pilihan mood (seperti *Excited*, *Joyful*, dsb.), aplikasi memuat file gambar .png baru dari folder *img/emotions/*. Proses pemuatan gambar berulang kali ini terlihat pada grafik memori Heap di VisualVM.
- Pembaruan Grafik Dinamis: Setiap kali mood dicatat atau diubah, *createMoodGraphWidget* melakukan kueri ulang untuk 7 hari terakhir dan merender ulang *AreaChart*. Proses kalkulasi skor mood dan rendering grafik ini adalah penyebab utama kenaikan CPU usage pada bagian dashboard.

Unit Testing (Bonus)

Terapkan unit testing pada tugas besar Anda. Unit testing dapat berupa apa saja. Contoh library yang dapat digunakan adalah JUnit pada Java. Jika mengerjakan bonus ini, kelompok harus membuat minimal 1 unit test untuk fitur yang dibuat pada pembagian sebelumnya.



1. Implementasi Library yang Sesuai

Pada bagian kode, terlihat penggunaan import org.junit.jupiter... dan import static org.junit.jupiter.api.Assertions.*. Ini dapat membuktikan menggunakan JUnit 5 (Jupiter), yang merupakan library standar dan disarankan dalam instruksi tugas tersebut.

2. Memenuhi Batas Minimal Unit Test

Instruksi meminta pembuatan minimal 1 unit test untuk fitur yang dibuat sebelumnya.

Dalam screenshot panel Testing, terlihat kamu memiliki 6/6 test yang berhasil dijalankan (Passed).

Hal tersebut melampaui standar minimal dengan menyediakan beberapa skenario pengujian, seperti testGetEntriesByDate_WhenDateNull() dan testSearchEntries_WithKeyword().

3. Relevansi dengan Fitur yang Dikerjakan

File yang dibuat adalah EntryControllerTest.java.

Ini secara langsung menguji logika pada EntryController, yang merupakan bagian inti dari fitur manajemen jurnal (seperti filter tanggal dan pencarian kata kunci) yang telah kamu selesaikan pada pembagian tugas sebelumnya.

4. Struktur Proyek yang Benar

Terlihat pada struktur folder VS Code, kamu meletakkan file test di bawah folder test/java/..., terpisah dari kode utama di folder src/main.

Pemisahan ini mengikuti best practice pengembangan aplikasi Java profesional dan mempermudah verifikasi oleh asisten dosen.

5. Bukti Eksekusi (Passed)

Panel Testing menunjukkan ikon centang hijau pada semua metode test.

Ini adalah bukti konkret bahwa unit test bukan sekadar kode pajangan, melainkan berfungsi dengan benar untuk memvalidasi keakuratan logika fitur yang kamu buat